Practices **Trigger Examples** in Microsoft SQL Server (MSSQL) for training, labs, and real business use cases.

What is a Trigger?

A **Trigger** is a special stored procedure that automatically runs when an event occurs in a database table.

Common Trigger Events

- INSERT
- UPDATE
- DELETE

Types of Triggers in MSSQL

Trigger Type	Purpose
AFTER Trigger	Runs after INSERT/UPDATE/DELETE
INSTEAD OF Trigger	Replaces the actual action
DDL Trigger	Monitors CREATE, ALTER, DROP
LOGON Trigger	Controls user login events

---

1. AFTER INSERT Trigger Example Banking Use Case — Audit New Customer

```sql
-- Create Database
CREATE DATABASE BankDB;
GO

USE BankDB;
GO

-- Customer Table
CREATE TABLE customers (
    customer_id INT PRIMARY KEY IDENTITY(1,1),
    customer_name VARCHAR(100),
    email VARCHAR(100),
    created_at DATETIME DEFAULT GETDATE()
);

-- Audit Table
CREATE TABLE customer_audit (
    audit_id INT PRIMARY KEY IDENTITY(1,1),
    customer_id INT,
    action_type VARCHAR(50),
    action_time DATETIME
);
GO
```

```sql
-- Trigger
CREATE TRIGGER trg_after_insert_customer
ON customers
AFTER INSERT
AS
BEGIN
    INSERT INTO customer_audit (
        customer_id,
        action_type,
        action_time
    )
    SELECT
        customer_id,
        'NEW CUSTOMER INSERTED',
        GETDATE()
    FROM inserted;
END;
GO

-- Test
INSERT INTO customers(customer_name, email)
VALUES ('Aung Aung', 'aung@gmail.com');
```

```sql
-- Check Result
SELECT * FROM customer_audit;
```

100 % 1 0

Results Messages

	audit_id	customer_id	action_type	action_time
1	1	1	NEW CUSTOMER INSERTED	2026-06-03 20:59:00.820

---

2. AFTER UPDATE Trigger Example Telecom Use Case — Track Package Changes

```sql
USE BankDB;
GO

-- Subscriber Table
CREATE TABLE subscribers (
    subscriber_id INT PRIMARY KEY IDENTITY(1,1),
    subscriber_name VARCHAR(100),
    package_name VARCHAR(50)
);

-- Package History
CREATE TABLE package_history (
    history_id INT PRIMARY KEY IDENTITY(1,1),
    subscriber_id INT,
    old_package VARCHAR(50),
    new_package VARCHAR(50),
    changed_time DATETIME
);
GO
```

```sql
-- Trigger
CREATE TRIGGER trg_after_update_package
ON subscribers
AFTER UPDATE
AS
BEGIN
    INSERT INTO package_history (
        subscriber_id,
        old_package,
        new_package,
        changed_time
    )
    SELECT
        d.subscriber_id,
        d.package_name,
        i.package_name,
        GETDATE()
    FROM deleted d
    INNER JOIN inserted i
        ON d.subscriber_id = i.subscriber_id;
END;
GO
```

```sql
-- Test Data
INSERT INTO subscribers(subscriber_name, package_name)
VALUES ('Su Su', 'Basic');

-- Update
UPDATE subscribers
SET package_name = 'Premium'
WHERE subscriber_id = 1;

-- Check History
SELECT * FROM package_history;
```

100 % 1 0

Results Messages

	history_id	subscriber_id	old_package	new_package	changed_time
1	1	1	Basic	Premium	2026-06-03 21:02:45.023

3. AFTER DELETE Trigger Example HR Use Case — Employee Delete Log

```
USE BankDB;
GO

-- Employee Table
CREATE TABLE employees (
    emp_id INT PRIMARY KEY IDENTITY(1,1),
    emp_name VARCHAR(100),
    department VARCHAR(50)
);

-- Delete Log Table
CREATE TABLE employee_delete_log (
    log_id INT PRIMARY KEY IDENTITY(1,1),
    emp_id INT,
    emp_name VARCHAR(100),
    deleted_time DATETIME
);
GO
```

```
-- Trigger
CREATE TRIGGER trg_after_delete_employee
ON employees
AFTER DELETE
AS
BEGIN
    INSERT INTO employee_delete_log (
        emp_id,
        emp_name,
        deleted_time
    )
    SELECT
        emp_id,
        emp_name,
        GETDATE()
    FROM deleted;
END;
GO
```

```sql
-- Test
INSERT INTO employees(emp_name, department)
VALUES ('Kyaw Kyaw', 'IT');

DELETE FROM employees
WHERE emp_id = 1;

-- Check Log
SELECT * FROM employee_delete_log;
```

100 % 1 0

Results Messages

	log_id	emp_id	emp_name	deleted_time
1	1	1	Kyaw Kyaw	2026-06-03 21:10:06.437

---

4. INSTEAD OF DELETE Trigger Example

Prevent Important Data Deletion

```sql
USE BankDB;
GO

CREATE TABLE protected_accounts (
    account_id INT PRIMARY KEY,
    account_name VARCHAR(100),
    balance MONEY
);
GO

-- Trigger
CREATE TRIGGER trg_prevent_delete
ON protected_accounts
INSTEAD OF DELETE
AS
BEGIN
    PRINT 'DELETE operation is not allowed.';
END;
GO

-- Test Data
INSERT INTO protected_accounts
VALUES (1, 'VIP Customer', 1000000);
```

```sql
-- Try Delete
DELETE FROM protected_accounts
WHERE account_id = 1;
```

100 % 1 0

Messages

```
DELETE operation is not allowed.
```

```sql
-- Check Data
SELECT * FROM protected_accounts;
```

100 % 1 0

Results Messages

	account_id	account_name	balance
1	1	VIP Customer	1000000.00

---

5. DDL Trigger Example Prevent Table Deletion

```sql
USE BankDB;
GO

CREATE TRIGGER trg_prevent_drop_table
ON DATABASE
FOR DROP_TABLE
AS
BEGIN
    PRINT 'Dropping tables is not allowed!';
    ROLLBACK;
END;
GO
```

---

6. Trigger with Validation

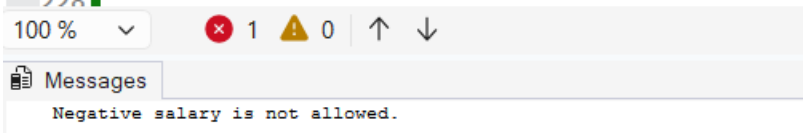
Prevent Negative Salary

```sql
USE BankDB;
GO

CREATE TABLE staff (
    staff_id INT PRIMARY KEY IDENTITY(1,1),
    staff_name VARCHAR(100),
    salary MONEY
);
GO

CREATE TRIGGER trg_validate_salary
ON staff
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT *
        FROM inserted
        WHERE salary < 0
    )
    BEGIN
        PRINT 'Negative salary is not allowed.';
        ROLLBACK TRANSACTION;
    END
END;
GO
```

```sql
-- Test
INSERT INTO staff(staff_name, salary)
VALUES ('Mg Mg', -50000);
```



Important MSSQL Trigger Tables

Virtual Table	Description
inserted	New data
deleted	Old data

Trigger Management Commands

View Triggers

```
229  SELECT name
230  FROM sys.triggers;
231
```

100 % 1 0

Results Messages

	name
1	trg_after_insert_customer
2	trg_after_update_package
3	trg_after_delete_employee
4	trg_prevent_delete
5	trg_prevent_drop_table
6	trg_validate_salary

Disable Trigger

```
232  DISABLE TRIGGER trg_validate_salary
233  ON staff;
```

Enable Trigger

```
235  ENABLE TRIGGER trg_validate_salary
236  ON staff;
```

Drop Trigger

```
238  DROP TRIGGER trg_validate_salary;
```
